

ANN and DL (CAI-361)

Objectives:

- Vision Transformers (ViT)
 - Categorization of Transformer models
 - Bert
 - GPT
-

Vision Transformers (ViT)

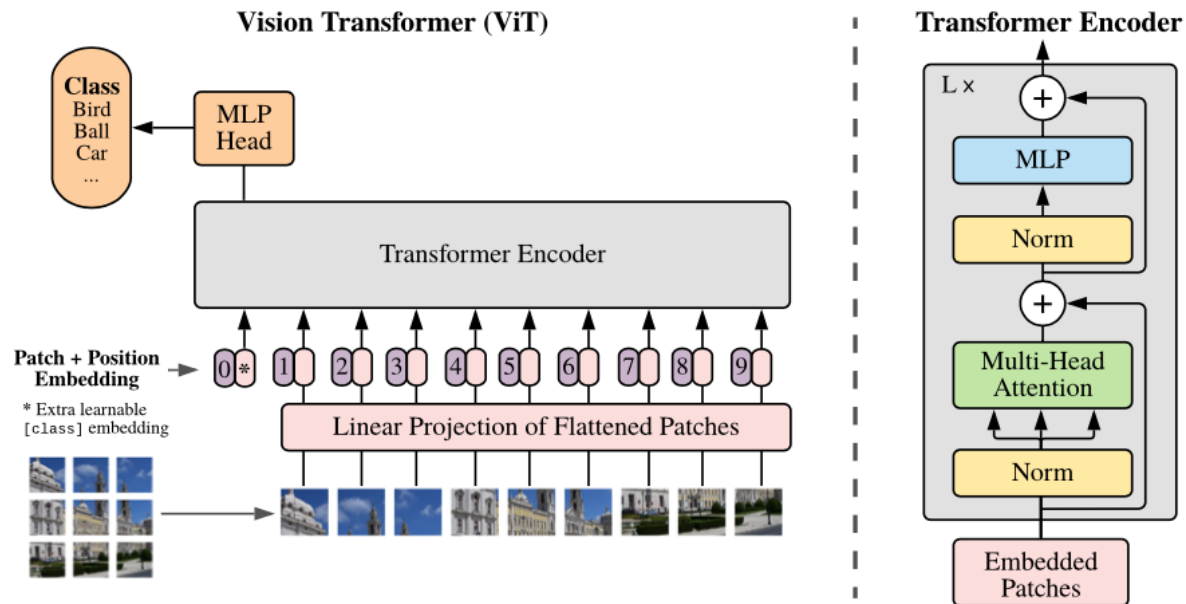
A vision transformer (ViT) is a transformer designed for computer vision. A ViT decomposes an input image into a series of patches (rather than text into tokens), serializes each patch into a vector, and maps it to a smaller dimension with a single matrix multiplication. These vector embeddings are then processed by a transformer encoder as if they were token embeddings.

ViTs were designed as alternatives to convolutional neural networks (CNNs). Compared to CNNs, ViTs are less data efficient, but have higher capacity.

Transformers measure the relationships between pairs of input tokens (words in the case of text strings), termed attention. The cost is quadratic in the number of tokens. For images, the basic unit of analysis is the pixel. However, computing relationships for every pixel pair in a typical image is prohibitive in terms of memory and computation. Instead, ViT computes relationships among pixels in various small sections of the image (e.g., 16x16 pixels), at a drastically reduced cost. The sections (with positional embeddings) are placed in a sequence. The embeddings are learnable vectors. Each section is arranged into a linear sequence and multiplied by the embedding matrix. The result, with the position embedding is fed to the transformer.

Transformers were introduced in Attention Is All You Need (2017). In 2020, an encoder-only Transformer was adapted for computer vision, yielding the ViT, which reached state of the art in image classification, overcoming the previous dominance of CNN.

ViT Architecture:



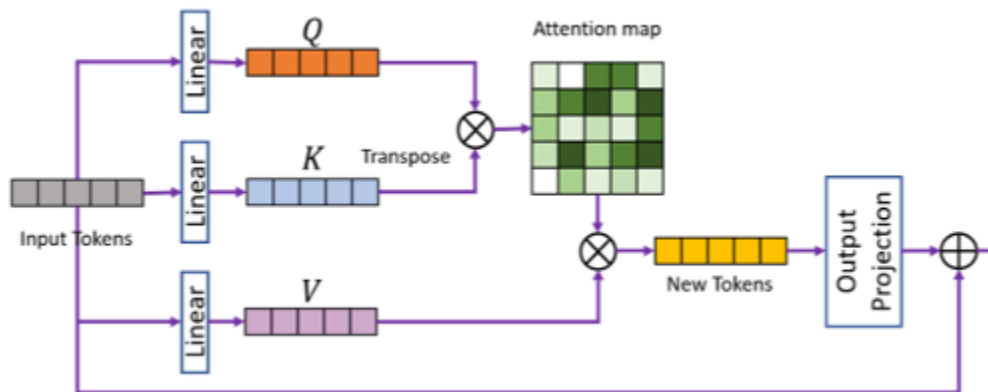
The concept of self-attention has been adapted for processing images with the use of Vision Transformers. Unlike text data, images are inherently two-dimensional, comprising pixels arranged in rows and columns. To address this challenge, ViTs convert images into sequences that can be processed by the Transformer.

- **Split an image into patches:** The first step in processing an image with a Vision Transformer is to divide it into smaller, fixed-size patches. Each patch represents a local region of the image.
- **Flatten the patches:** Within each patch, the pixel values are flattened into a single vector. This flattening process allows the model to treat image patches as sequential data.
- **Produce lower-dimensional linear embeddings:** These flattened patch vectors are then projected into a lower-dimensional space using trainable linear transformations. This step reduces the dimensionality of the data while preserving important features.
- **Add positional encodings:** To retain information about the spatial arrangement of the patches, positional encodings are added. These encodings help the model understand the relative positions of different patches in the image.
- **Feed the sequence into a Transformer encoder:** The input to a standard Transformer encoder comprises the sequence of patch embeddings and positional embeddings. This encoder is composed of multiple layers, each containing two critical components: multi-head self-attention mechanisms (MSPs), responsible for calculating attention weights to prioritize input sequence elements during predictions, and multi-layer perceptron (MLP) blocks. Before each block, layer normalization (LN) is applied to appropriately scale and center the data within the layer, ensuring stability and efficiency during training. During

the training, an optimizer is also used to adjust the model's hyperparameters in response to the loss computed during each training iteration.

- **Classification Token:** To enable image classification, a special "classification token" is prepended to the sequence of patch embeddings. This token's state at the output of the Transformer encoder serves as the representation of the entire image.

Attention Module of ViT



Applications of Vision Transformers

- Image Classification: Classifying images into predefined categories
- Object Detection: Identifying and localizing objects within images
- Semantic Segmentation: Assigning a label to each pixel in an image to identify object boundaries
- Image Generation: Generating new images based on a given context or description

Limitations of Vision Transformers

- Large Datasets: Training Vision Transformers effectively often requires large datasets, which might not be available for all domains.
- Computational Demands: Training ViT can be computationally intensive due to the self-attention mechanisms.
- Spatial Information: ViT's sequential processing might not capture fine-grained spatial patterns as effectively as CNNs for tasks like segmentation.

Categorization of transformer models

Transformers ensure the efficiency of LLMs in processing information. Their role is critical to ensure improved accuracy, faster training on data, and wider applicability. Hence, it is important to understand the different model types available to choose the right one for your needs. However, before we delve into the many types of transformer models, it is important to understand the basis of their classification.

Architecture-based classification

The most fundamental categorization of transformer models is done based on their architecture. The variations are designed to perform specific tasks or cater to the limitations of the base architecture. The very common model types under this category include encoder-only, decoder-only, and encoder-decoder transformers. The general architecture of transformer models are

1) Encoder-only transformer

As the name suggests, this architectural type uses only the encoder part of the transformer, focusing on encoding the input sequence. For this model type, understanding the input sequence is crucial while generating an output sequence is not required.

Some common applications of an encoder-only transformer include:

Text classification

It is focused on classifying the input data based on defined parameters. It is often used in email spam filters to categorize incoming emails. The transformer model can also train over the patterns for effective filtration of unwanted messages.

Sentimental analysis

This feature makes it an appropriate choice for social media companies to analyze customer feedback and their emotion toward a service or product. It provides useful data insights, leading to the creation of effective strategies to enhance customer satisfaction.

Anomaly detection

It is particularly useful for finance companies. The analysis of financial transactions allows the timely detection of anomalies. Hence, possible fraudulent activities can be addressed promptly. Other uses of an encoder-only transformer include question-answering, speech recognition, and image captioning.

2) Decoder-only transformer

It is a less common type of transformer model that uses only the decoder component to generate text sequences based on input prompts. The self-attention mechanism allows the model to focus on previously generated outputs in the sequence, enabling it to refine the output and create more contextually aware results. Some common uses of decoder-only transformers include:

Text summarization

It can iteratively generate textual summaries of the input, focusing on including the important aspects of information.

Text generation

It builds on a provided prompt to generate relevant textual outputs. The results cover a diverse range of content types, like poems, codes, and snippets. It is capable of iterating the process to create connected and improved responses.

Chatbots

It is useful to handle conversational interactions via chatbots. The decoder can also consider previous conversations to formulate relevant responses.

3) Encoder-decoder Transformer

This is a classic architectural type of transformer, efficiently handling sequence-to-sequence tasks, where you need to transform one type of sequence (like text) into another (like a

translation or summary). An encoder processes the input sequence while a decoder is used to generate an output sequence.

Some common uses of an encoder-decoder transformer include:

Machine translation

Since the sequence is important at both the input and output, it makes this transformer model a useful tool for translation. It also considers contextual references and relationships between words in both languages.

Text summarization

While this use overlaps with that of a decoder-only transformer, text summarization differs from an encoder-decoder transformer due to its focus on the input sequence. It enables the creation of summaries that focus on relevant aspects of the text highlighted in an input prompt.

Question-answering

It is important to understand the question before providing a relevant answer. An encoder-decoder transformer allows this focus on both ends of the communication, ensuring each question is understood and answered appropriately.

Categorization based on pre-training approaches

While architecture is a basic component of consideration, the training techniques are equally crucial components for transformers. Pre-training approaches refer to the techniques used to train a transformer on a general dataset before finetuning it to perform specific tasks. Some common approaches that define classification under this category include Masked Language Models (MLMs), autoregressive models, and conditional transformers. While the architectural differences provide a basis for transformer types, the models can be further classified based on their techniques of pre-training. Let's explore the various transformer models segregated based on pre-training approaches.

1) Masked Language Models (MLMs)

Models with this pre-training approach are usually encoder-only in architecture. They are trained to predict a masked word in a sentence based on the contextual information of the surrounding words. The training enables these model types to become efficient in understanding language relationships.

Some common MLM applications are:

Boosting downstream NLP tasks

MLMs train on massive datasets, enabling the models to develop a strong understanding of language context and relationships between words. This knowledge enables MLM models to contribute and excel in diverse NLP applications.

General-purpose NLP tool

The enhanced learning, knowledge, and adaptability of MLMs make them a part of multiple NLP applications. Developers leverage this versatility of pre-trained MLMs to build a basis for different NLP tools.

Efficient NLP development

The pre-trained foundation of MLMs reduces the time and resources needed for the deployment of NLP applications. It promotes innovation, faster development, and efficiency.

2) Autoregressive models

Typically built using a decoder-only architecture, this pre-training model is used to generate sequences iteratively. It can predict the next word based on the previous one in the text you have written. Some common uses of autoregressive models include:

Text generation

The iterative prediction from the model enables it to generate different text formats. From codes and poems to musical pieces, it can create all while iteratively refining the output as well.

Chatbots

The model can also be utilized in a conversational environment, creating engaging and contextually relevant responses,

Machine translation

While encoder-decoder models are commonly used for translation tasks, some languages with complex grammatical structures are supported by autoregressive models.

3) Conditional transformer

This transformer model incorporates the additional information of a condition along with the main input sequence. It enables the model to generate highly specific outputs based on particular conditions, ensuring more personalized results.

Some uses of conditional transformers include:

Machine translation with adaptation

The conditional aspect enables the model to set the target language as a condition. It ensures better adjustment of the model to the target language's style and characteristics.

Summarization with constraints

Additional information allows the model to generate summaries of textual inputs based on particular conditions.

Speech recognition with constraints

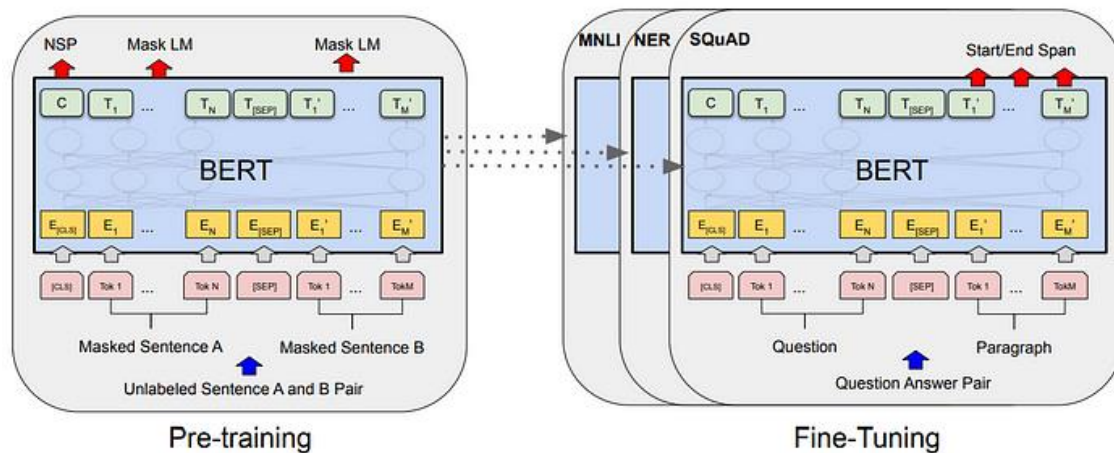
With the consideration of additional factors like speaker ID or background noise, the recognition process enhances to produce improved results.

BERT

BERT stands for **Bidirectional Encoder Representations from Transformers**. BERT consists of two parts: **Pre-training and Fine-tuning**. BERT is designed to pre-train deep bidirectional representations from the unlabeled text by jointly conditioning on both left and right contexts in all layers. Then the pre-trained BERT model can be fine-tuned with just one additional output layer to create state-of-the-art models for a wide range of tasks. A distinctive feature of BERT is its unified architecture across different tasks, such as name entity recognition, sentence classification, and question answering.

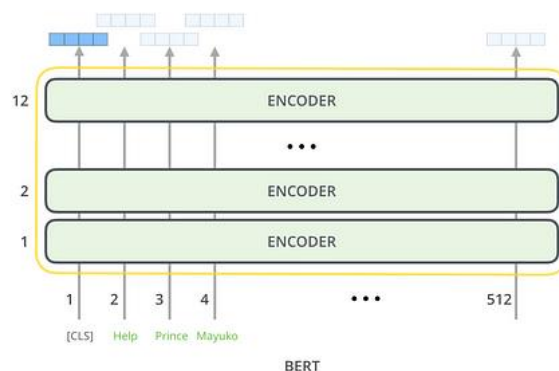
Apart from output layers, the same architectures are used in both pre-training and fine-tuning. The same pre-trained model parameters are used to initialize models for different downstream tasks. During fine-tuning, all parameters are fine-tuned. The input can be one sentence (for name entity recognition) or a pair of sentences (for question answering). Throughout this work, a “sentence” can be an arbitrary span of contiguous text rather than an actual linguistic sentence. A “sequence”

refers to the input token sequence to BERT, which may be a single or two sentences packed together. In front of every input, a special symbol [CLS] is added. If the input is a pair, a special separator token [SEP] is inserted in between (e.g., separating questions/answers).

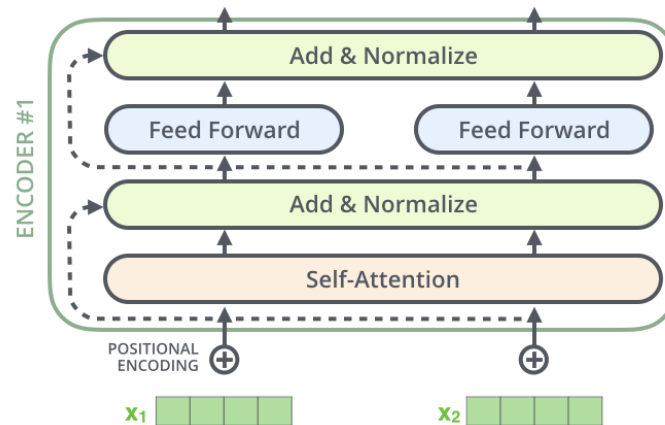


Architecture

BERT is a trained Transformer Encoder stack. The first input token is supplied with a special [CLS] token. CLS here stands for classification. Each position outputs a vector of size *hidden_size* (768 in BERT Base). For the sentence classification example, we focus on the output of only the first position (that we passed the special [CLS] token).



Just like the encoder of the transformer, BERT takes a sequence of words as input which keep flowing up the stack. Each layer applies self-attention, passes its results through a feed-forward network, and then hands it off to the next encoder.



Input representation

To make BERT handle a variety of downstream tasks, the input representation is able to unambiguously represent both a single sentence and a pair of sentences (e.g., Question, Answer) in one token sequence. Its input representation is constructed for a given token by summing the corresponding **token embeddings**, **segment embeddings**, and **position embeddings**. The input embeddings are the sum of the token embeddings, the segmentation embeddings, and the position embeddings.

Input	[CLS]	my	dog	is	cute	[SEP]	he	likes	play	##ing	[SEP]
Token Embeddings	$E_{[CLS]}$	E_{my}	E_{dog}	E_{is}	E_{cute}	$E_{[SEP]}$	E_{he}	E_{likes}	E_{play}	$E_{##ing}$	$E_{[SEP]}$
	+	+	+	+	+	+	+	+	+	+	+
Segment Embeddings	E_A	E_A	E_A	E_A	E_A	E_A	E_B	E_B	E_B	E_B	E_B
	+	+	+	+	+	+	+	+	+	+	+
Position Embeddings	E_0	E_1	E_2	E_3	E_4	E_5	E_6	E_7	E_8	E_9	E_{10}

Pre-training

BERT does not use traditional left-to-right or right-to-left language models to pre-train BERT. Instead, we pre-train BERT using two unsupervised tasks: Masked LM and Next sentence prediction.

Task 1: Masked LM (Language Modeling)

We randomly mask some percentage of the input tokens and then predict those masked tokens. This procedure is called a “masked LM” (MLM). A [MASK] token is used to replace the masked words. A downside is that we are creating a mismatch between pre-training and fine-tuning since the [MASK] token does not appear during fine-tuning. To mitigate this, we do not always replace “masked” words with the actual [MASK] token. Instead, we reserve randomly sample some tokens to be replaced with a random word or keep them unchanged:

- 80% of the time: Replace the word with the [MASK] token, e.g., my dog is hairy → my dog is [MASK]
- 10% of the time: Replace the word with a random word, e.g., my dog is hairy → my dog is apple
- 10% of the time: Keep the word unchanged, e.g., my dog is hairy → my dog is hairy. The purpose of this is to bias the representation towards the actual observed word.

Task 2: Next Sentence Prediction (NSP)

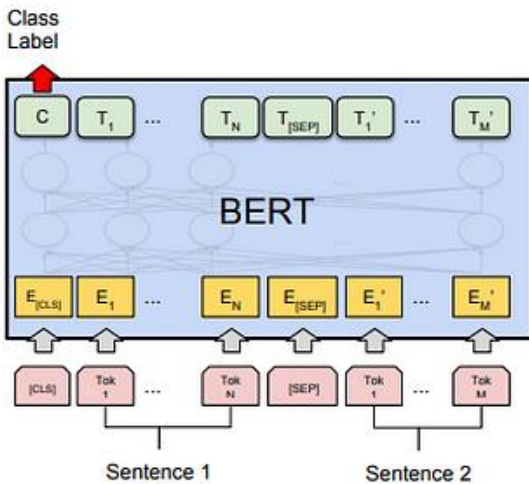
Many important downstream tasks are based on understanding the relationship between two sentences, such as Question Answering (QA) and Natural Language Inference (NLI), which language modeling does not capture directly. BERT pre-trains for a binarized next sentence prediction task.

To generate each training input sequence, we sample two spans of text from the corpus, which we refer to as “sentences” even though they are typically much longer than single sentences (but can be shorter also). The first sentence receives the A embedding, and the second receives the B embedding. 50% of the time B is the actual next sentence that follows A, and 50% of the time, it is a random sentence, which is done for the “next sentence prediction” task. The training loss is the sum of the mean **masked LM likelihood** and the mean **next sentence prediction likelihood**.

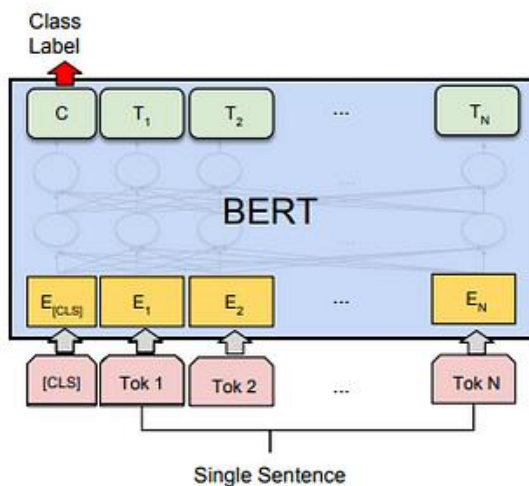
Fine-tuning

For each task, we plug in the task-specific inputs and outputs into BERT and fine-tune all the parameters end-to-end. At the input, sentence A and sentence B from pre-training are analogous to (1) sentence pairs in paraphrasing, (2) hypothesis-premise pairs in entailment, (3) question-passage pairs in question answering, and (4) a degenerate text- $\emptyset$ pair in text classification or sequence tagging. At the output, the **token representations are fed into an output layer** for token level tasks, such as sequence tagging or question answering. The **[CLS] representation is fed into an output layer for classification**, such as entailment or sentiment analysis.

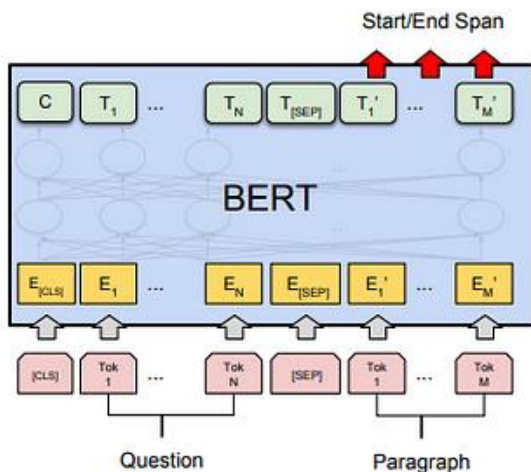
The following figure shows how to adapt the same BERT model for different tasks:



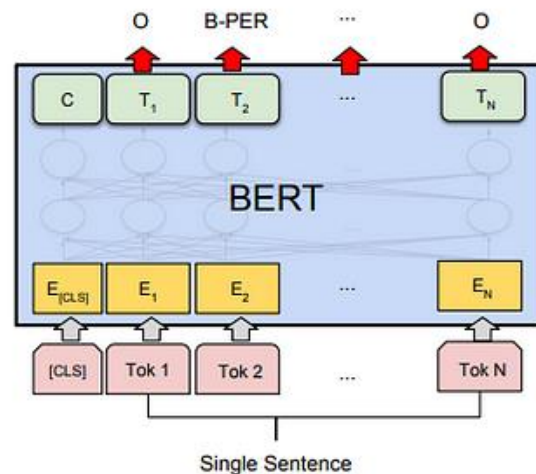
(a) Sentence Pair Classification Tasks:
MNLI, QQP, QNLI, STS-B, MRPC,
RTE, SWAG



(b) Single Sentence Classification Tasks:
SST-2, CoLA



(c) Question Answering Tasks:
SQuAD v1.1



(d) Single Sentence Tagging Tasks:
CoNLL-2003 NER

The bench datasets include:

MNLI: Multi-Genre Natural Language Inference is a large-scale, crowdsourced entailment classification task (Williams et al., 2018). Given a pair of sentences, the goal is to predict whether the second sentence is an entailment, contradiction, or neutral with respect to the first one.

QQP: Quora Question Pairs is a binary classification task that aims to determine if two questions asked on Quora are semantically equivalent.

SST-2: The Stanford Sentiment Treebank is a binary single-sentence classification task consisting of sentences extracted from movie reviews with human annotations of their sentiment.

SQuAD: The Stanford Question Answering Dataset (SQuAD v1.1) is a collection of 100k crowdsourced question/answer pairs.

CoNLL-2003: Named Entity Recognition (NER) task.

GPT

The Generative Pre-trained Transformer (GPT) is a model, developed by Open AI to understand and generate human-like text. GPT has revolutionized how machines interact with human language, enabling more intuitive and meaningful communication between humans and computers. Originally released on 11th June 2018, GPT has undergone transformations in recent years. There is no better model to begin discussing transformers than GPT, which stands for Generative Pre-Training. It pioneered using unsupervised learning for pre-training and supervised learning for fine-tuning, which is currently commonly employed by many transformers.

It was trained on a book corpus dataset consisting of 7,000 unpublished books. The architecture of GPT consists of 12 decoders stacked together, meaning it is a decoder-only model. GPT-1 consists of 117 million parameters, a small number compared to the transformers developed today! GPT was just the beginning of the era of transformers.

Text Preparation

Before we get to the diagram, let's first understand how text input is prepared. The raw text has to be tokenized, which means converting words into integers that map to indices in the model's vocabulary. GPT-2 uses a process called Byte-Pair Encoding (BPE) for tokenization, which breaks text down into subwords. You can use Tiktoken, a fast BPE tokenizer used in OpenAI's models. This turns the text into a list of integers representing subwords or words from GPT-2's vocabulary. We also append a special `<|endoftext|>` token to mark the end of the text sequence.

Next, we will rearrange the tokens into a format the GPT model can process. The model expects the input data to be in the form of a batch of sequences. The input tensor should have the shape (B, T), where:

- B is the batch size (how many sequences we process in parallel).
- T is the sequence length (number of tokens in each sequence).

Here, x contains the input tokens, and y contains the target tokens, shifted by one position so that the model can learn to predict the next token in the sequence. During training, these tensors are fed into the model, with the target tensor y used to calculate the loss using cross-entropy loss.

Finally, let's take a look at the GPT-2 architectural diagram to better understand how the input data moves through the model. Below are some key configuration parameters for GPT-2:

- Vocabulary Size (V): 50,257
- Maximum Sequence Length (T): 1024
- Embedding Dimensionality (C): 768
- Number of Heads (h): 12
- Number of Layers (N): 12
- Batch Size (B): 512

